

CSE273: Foundations of Machine Learning

UNIT 1 (Part-2): NumPy for Numerical Computing

NumPy:

- NumPy (Numerical Python)
- Used for numerical computing in Python
- Efficient handling of arrays & matrices

NumPy Arrays:

A powerful n-dimensional data container, similar to lists but optimized for speed.

Why NumPy?

- Faster than Python lists
- Supports multi-dimensional arrays
- Used in ML, Data Science, AI

NumPy Arrays:

- Homogeneous data type
- Stored in contiguous memory
- ndarray is core object

Creating Arrays:

- array()
→ `a = np.array([1, 2, 3, 4])`
- arange()
→ `np.arange(0, 10, 2)`
- zeros(), ones()
→ `np.zeros(5), np.zeros((2, 3)), np.ones((3, 2))`
- full()
→ `np.full((3, 2), 7)`
- linspace(): Creates **evenly spaced values** between a range.
→ `np.linspace(0, 1, 5)`
- eye:
→ `np.eye(4)`

2D array and 3D array:

```
b=np.array([[1,2],[7,8]])
print(b)
```

```
a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
```

Method	Purpose
array()	Convert list to array
arange()	Range-based values
zeros()	Initialize with 0
ones()	Initialize with 1
full()	Constant value
linspace()	Equal spacing
eye()	Identity matrix

Task:

```
import numpy as np
a=np.array([1,"b",3])
print(a)
print(a.dtype)
```

N-Dimensional Array:

shape = (3,2)

Diagram illustrating a 2D array with shape (3,2). The array is a 3x2 grid of zeros. The width is labeled 2 and the height is labeled 3.

0	0
0	0
0	0

`np.zeros(shape)`

1	1
1	1
1	1

`np.ones(shape)`

7	7
7	7
7	7

`np.full(shape,7)`

Array Properties:

- ndim – dimensions
- shape – rows, columns
- size – total elements
- dtype – data type

Indexing & Slicing:

- Access elements using index
- Supports slicing like lists
- Boolean indexing

N-Dimensional Array:

```
npr = np.array([[1, 2],
                [4, 5],
                [7, 8]])
```

$npr[1,0] = 4$

Diagram illustrating the indexing of the 2D array `npr`:

The array is represented as a 3x2 grid:

0	1	2
1	4	5
2	7	8

Annotations:

- A downward arrow points from the first index '1' in `npr[1,0]` to the row index '1' in the grid.
- A rightward arrow points from the second index '0' in `npr[1,0]` to the column index '0' in the grid.
- The value 4 at row 1, column 0 is highlighted with a blue diagonal pattern.

Selection:

Diagram illustrating various selection operations on the 2D array `npr`:

Row Selection:

- `npr`: The full 3x2 array.
- `npr[:]`: All rows (0 to 2).
- `npr[0:3]`: Rows 0 to 2 (all rows).
- `npr[...]`: All elements.

Column Selection:

- `npr[:,0]`: The first column (values 1, 4, 7).

Row and Column Selection:

- `npr[0][1]`: Element at row 0, column 1 (value 2).
- `npr[1][0]`: Element at row 1, column 0 (value 4).

Excluded Selection:

Diagram showing the exclusion of the first row (row 0) from the selection:

`npr[1:]`: Rows 1 to 2 (values 4, 5 and 7, 8).

Advanced Selection:

- `npr[0][0:2][0:2]..`: Selection of the first row, first two columns, followed by an ellipsis.
- `npr[0][...]`: Selection of the first row, all columns.
- `npr[0][:]`: Selection of the first row, all columns.
- `npr[0][:][...]`: Selection of the first row, all columns, followed by an ellipsis.
- `:`: Selection of all elements.

Negative Indexing and Direction:

```

index      0    1    2    3    4    5    6    7    8    9    10
np.array(['S', 'O', 'L', 'O', 'T', 'H', 'O', 'U', 'G', 'H', 'T'])
index     -11  -10  -9   -8   -7   -6   -5   -4   -3   -2   -1

```

```
npr[::-2]
```

```
np.array(['S', 'O', 'L', 'O', 'T', 'H', 'O', 'U', 'G', 'H', 'T'])
```

Mathematical Operations:

- Element-wise operations
- Add, subtract, multiply, divide
- Broadcasting

Arithmetic Operations:

✓ Same Shape

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \\ 7 & 8 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 4 & 5 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 8 & 10 \\ 14 & 16 \end{bmatrix}$$

✓ Same Columns size

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \\ 7 & 8 \end{bmatrix} + \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 5 & 7 \\ 8 & 10 \end{bmatrix}$$

✓ Single element

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \\ 7 & 8 \end{bmatrix} + 2 = \begin{bmatrix} 3 & 4 \\ 6 & 7 \\ 9 & 10 \end{bmatrix}$$

axis=0 vs axis=1

```
▶ d = np.array([[1, 2, 3],
               [4, 5, 6], [3, 4, 4]])
n = d.sum(axis=0)
print(n)
```

... [8 11 13]

- ◆ axis = 0 → Column-wise operation
- 👉 Works down the rows

```
▶ d = np.array([[1, 2, 3],
               [4, 5, 6], [3, 4, 4]])
n = d.sum(axis=1)
print(n)
```

... [6 15 11]

- ◆ axis = 1 → Row-wise operation
- 👉 Works across the columns

Broadcasting:

- Broadcasting allows NumPy to perform arithmetic operations on arrays of different shapes.
- Instead of writing loops, NumPy automatically stretches the smaller array to match the shape of the larger array.

Broadcasting Rules:

- Compare shapes of two arrays from the trailing dimensions (rightmost first).
- Two dimensions are compatible if:
 - They are equal, or
 - One of them is 1
- If shapes are compatible, the array with dimension 1 is “stretched” to match the other array.
- If not compatible → Error.

python

```
a = np.array([1, 2, 3])
b = 10
print(a + b)
```

Output:

csharp

```
[11 12 13]
```

- Scalar 10 is broadcast to [10,10,10]

python

```
a = np.array([[1], [2], [3]]) # shape (3,1)
b = np.array([10,20,30])      # shape (3,)
print(a + b)
```

Step:

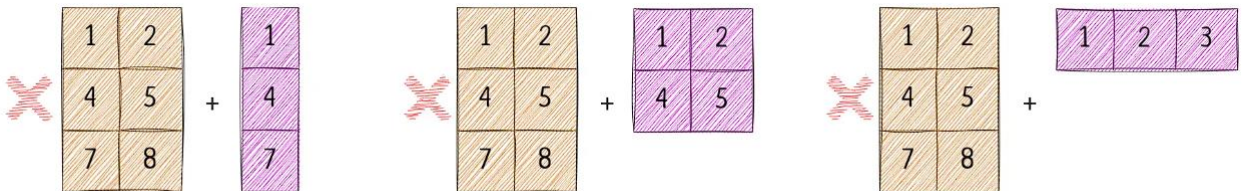
- a shape (3,1)
- b shape (3,) → treated as (1,3)
- Result shape (3,3)

Output:

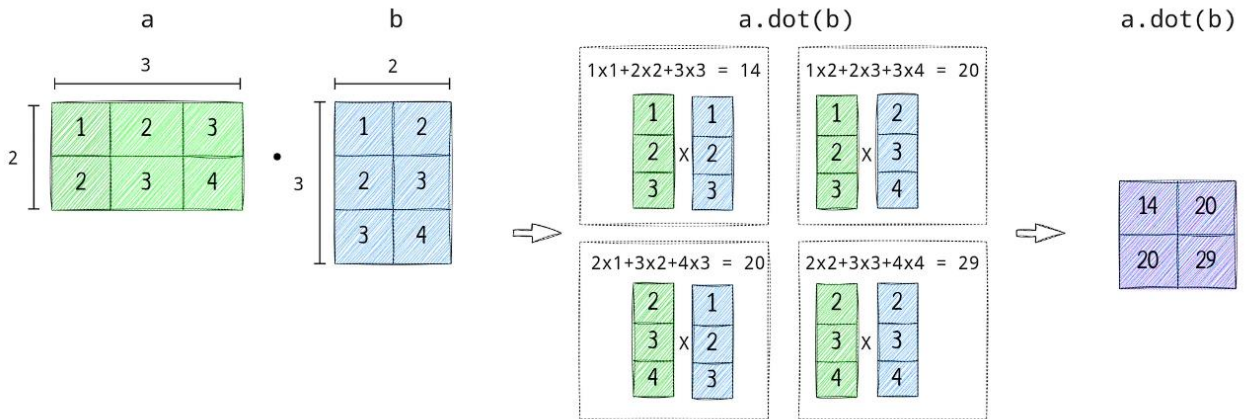
lua

```
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

Broadcasting:



Mathematical Operations:



What is the output of $A @ B$?

Statistical Functions:

- `mean()`, `median()`
- `std()`, `var()`
- `min()`, `max()`, `sum()`

Aggregation:

`npr.min()`



1	2
4	5
7	8

= 12

= 15

`npr.max()`

+
└──────────┘

`npr.sum()` = 27

Aggregation:

fn	Description
a.sum()	sum
a.min()	min
a.max()	max
a.cumsum()	Cumulative sum
a.mean()	Mean
a.median()	Median
a.corrcoef()	Correlation coefficient
np.std(a)	Standard deviation

Linear Algebra:

- dot()
- transpose()
- inverse()
- eigenvalues

Reshaping Arrays:

- `reshape()`
- `flatten()`
- `ravel()`

Feature	<code>ravel()</code>	<code>flatten()</code>
Returns	View (usually)	Copy
Modifying array	Can modify original	Original unchanged
Memory usage	Less (no copy)	More (copy)
Speed	Faster	Slightly slower

Restructuring of Array:

a

	3		
2	1	2	3
	4	5	6

Reshape

	2	
3	1	2
	3	4
	5	6

`a.reshape((3,2))`

Resize

1	2
3	4

`a.resize((2,2))`

Transpose

1	4
2	5
3	6

rows into cols

`a.T`

Flatten

1	2	3	4	5	6
---	---	---	---	---	---

`a.ravel()`

Appending and Concatenating Arrays:

a

1	2
3	4

b

1	2
3	4

`np.append(a,b)`

1	2	3	4	1	2	3	4
---	---	---	---	---	---	---	---

1	2
3	4
1	2
3	4

`np.concatenate((a,b))`
`np.concatenate((a,b), axis=0)`
`np.append(a,b, axis=0)`

1	2	1	2
3	4	3	4

`np.concatenate((a,b),axis=1)`
`np.append(a,b, axis=1)`

Matrix Operations:

```

M1 = np.array([[1,2],[3,4]])
M2 = np.array([[5,6],[7,8]])

print("M1:", M1, sep="\n")
print("M2:", M2, sep="\n")

print("Addition M1+M2:", M1+M2, sep="\n")
print("Subtraction M1-M2:", M1-M2, sep="\n")
print("Element-wise multiplication M1*M2:", M1*M2, sep="\n")
print("Matrix multiplication M1 @ M2:", M1 @ M2, sep="\n")
print("Transpose of M1:", M1.T, sep="\n")
print("Determinant of M1:", np.linalg.det(M1))
print("Inverse of M1:", np.linalg.inv(M1), sep="\n")
eigvals, eigvecs = np.linalg.eig(M1)
print("Eigenvalues of M1:", eigvals)
print("Eigenvectors of M1:", eigvecs, sep="\n")
print("Trace of M1:", np.trace(M1))

```

Trace of a square matrix is the sum of all its diagonal elements (from top-left to bottom-right).

M1:	
[[1 2]	
[3 4]]	
M2:	
[[5 6]	
[7 8]]	
Addition M1+M2:	
[[6 8]	
[10 12]]	
Subtraction M1-M2:	
[[-4 -4]	
[-4 -4]]	
Element-wise multiplication M1*M2:	
[[5 12]	
[21 32]]	
Matrix multiplication M1 @ M2:	
[[19 22]	
[43 50]]	
Transpose of M1:	
[[1 3]	
[2 4]]	
	Determinant of M1: -2.0000000000000004
	Inverse of M1:
	[[-2. 1.]
	[1.5 -0.5]]
	Eigenvalues of M1: [-0.37228132 5.37228132]
	Eigenvectors of M1:
	[[-0.82456484 -0.41597356]
	[0.56576746 -0.90937671]]
	Trace of M1: 5



Random Module: NumPy provides **numpy.random** to generate random numbers and arrays.

0.20978939	0.22842412
0.06289605	0.94502875
0.81051756	0.82297961

0.32606574	-0.44553714
0.10393725	-2.86527507
-0.28032726	-1.23097668

from the
"standard normal"
distribution sample

`np.random.random(shape)`

`np.random.randn(3,2)`

```
z = np.random.randint(1, 10, size=5)
print(z)
```



```
np.random.seed(10)
print(np.random.randint(1, 100, 5))
np.random.seed(10)
print(np.random.randint(1, 100, 5))
np.random.seed(10)
print(np.random.randint(1, 100, 5))
```

```
... [10 16 65 29 90]
     [10 16 65 29 90]
     [10 16 65 29 90]
```

**Task:**

An institute stores student exam scores for multiple subjects in a **NumPy array**. Each row represents a student, and each column represents a subject.

Goal: Analyze the dataset using NumPy operations to support academic decisions.

Student ID	Math	Science	English	Computer
Student 1	78	85	90	88
Student 2	92	81	76	89
Student 3	65	70	72	68
Student 4	88	90	94	92
Student 5	55	60	58	62

Questions:

1. Compute the average marks of each student using NumPy.
2. Calculate the average marks for each subject.
3. Find the total marks scored by each student.
4. Determine the maximum and minimum marks in the entire dataset.
5. Extract the marks of the 2nd student using slicing.



NumPy vs Python List:

- Faster computation
- Less memory
- Vectorized operations

Applications of NumPy:

- Machine Learning
- Image Processing
- Scientific Computing